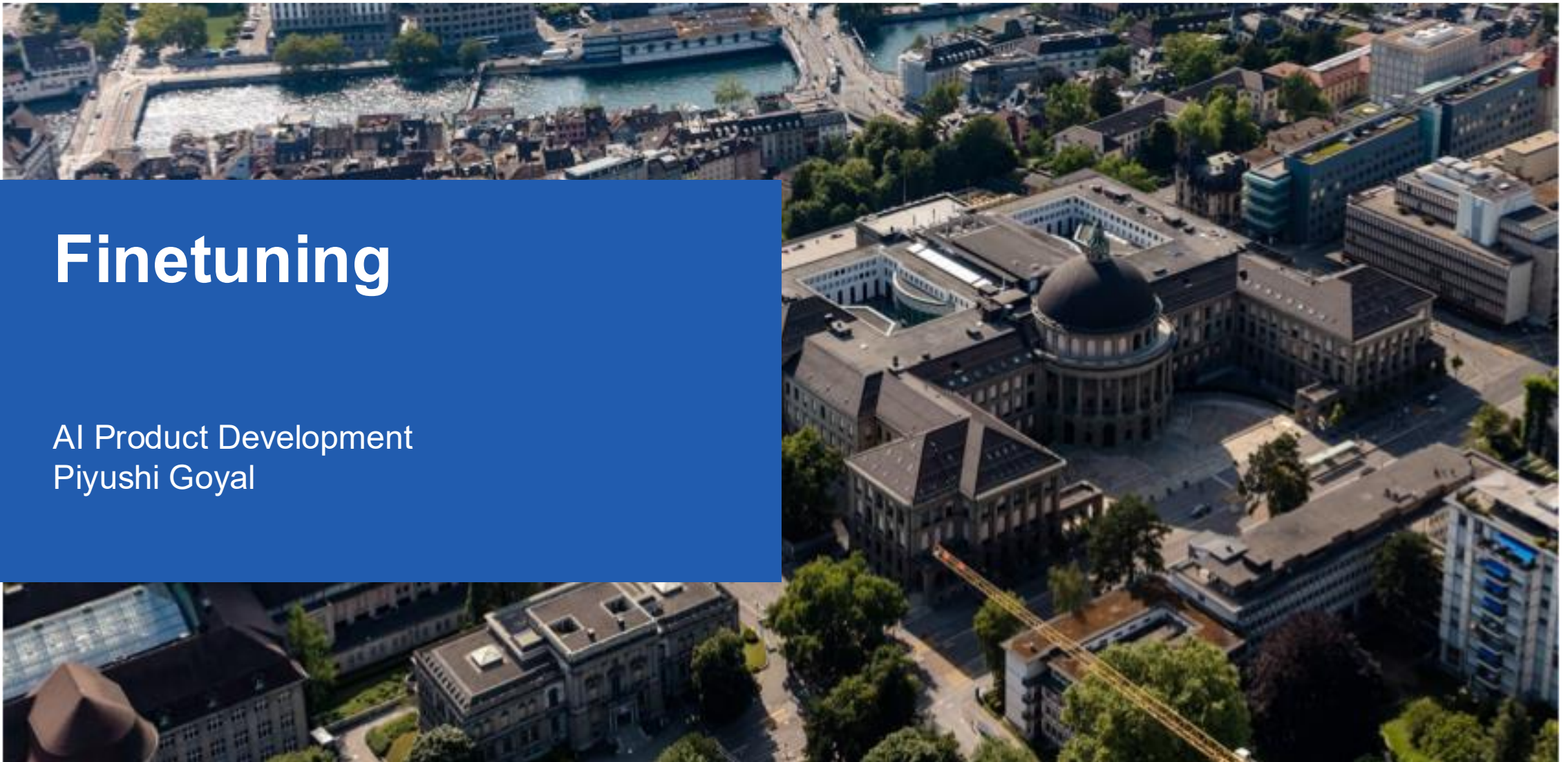
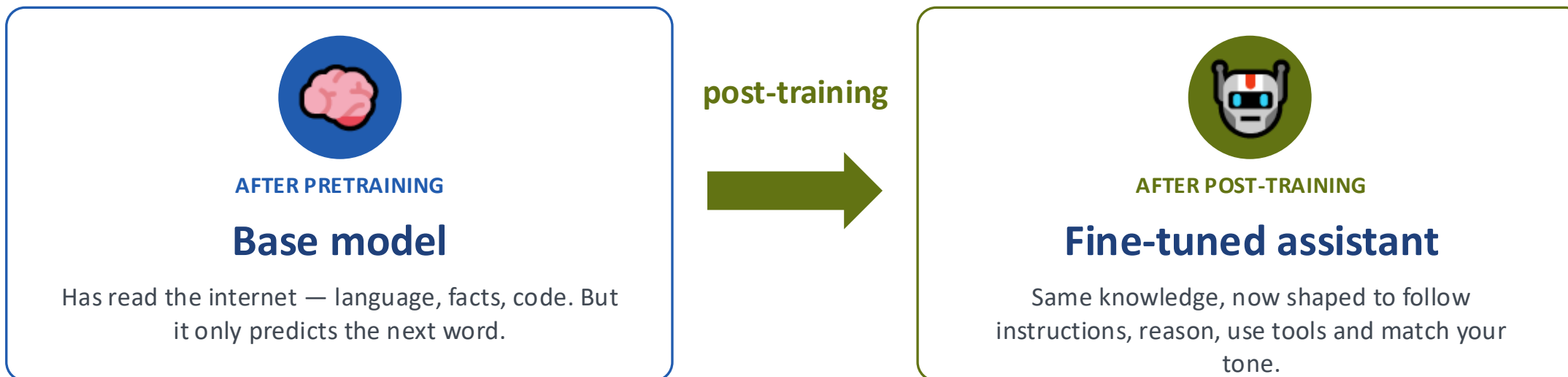


Finetuning

AI Product Development
Piyushi Goyal



Where fine-tuning actually lives



Same question to each: **"What is the capital of France?"**

Base model

*"What is the capital of Germany?
What is the capital of..."
just continues the pattern, never answers*

Fine-tuned assistant

"The capital of France is Paris."
answers the question you actually asked

Inside post-training: a menu



Supervised fine-tuning (SFT)

Show it (instruction → answer) pairs. It learns to follow instructions, not just autocomplete.



RL / RLHF

Let it try, then reward good replies.



Preference alignment

Rank answers better vs worse, so it stays helpful, honest and safe.



Tool-use & agents

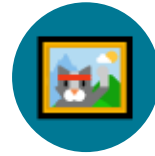
Teach it to search, call tools and run code.

This is why today's assistants can code, see and search



Claude — coding & agents

Post-trained hard on software: read a repo, write code, run tests, fix errors. It can act, not just chat.



Gemini — images & multimodal

Post-trained to read images, charts and screenshots, not only text.



DeepSeek-R1 — reasoning

Post-trained with reward to think step by step before answering.



Web-search & fresh answers: every mainstream chatbot was post-trained to know WHEN to look things up and fetch current information, instead of guessing from old training data.

PEFT and LoRA

PEFT

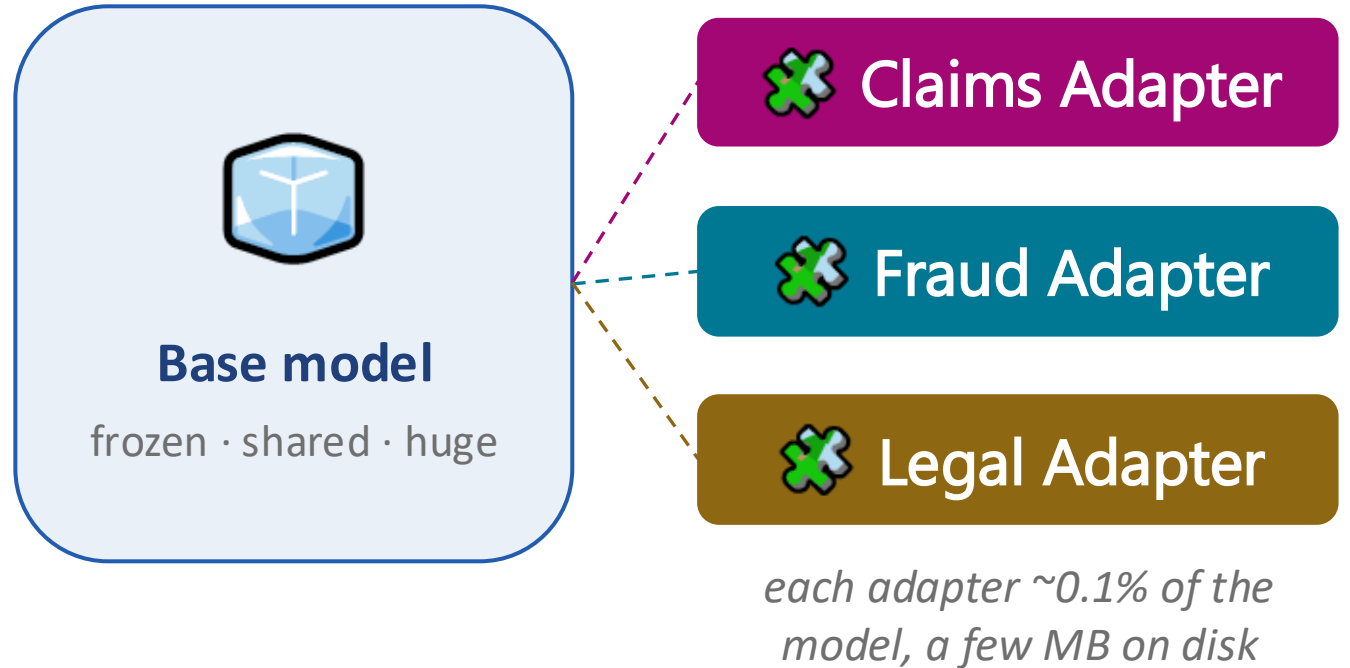
Parameter-Efficient Fine-Tuning

Change only a tiny slice of a giant model's weights. Same result, far less compute.

LoRA

Low-Rank Adaptation, the popular one

Freeze the big brain. Bolt on a small adapter. Only about 0.1% of the weights are trained.



PEFT and LoRA

Weight updates live in a low-dimensional subspace



Full Update Matrix ΔW

size: $d \times d$, very large

w_1	4_2	3_6	w_4	q_3
3_6	w_1	1	4_8	5
4_7	w_2	w_1	2_8	w_1
2_2	2_4	2_3	3_9	w_1
5_1	w_1	w_4	4	w_1

Most of this is noise.



high cost



expensive to store and train

LoRA insight



decompose into two small matrices

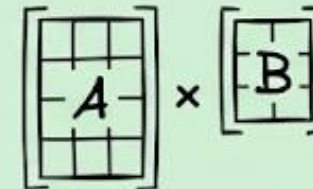
approximate, don't replace



low-rank is enough

$A \times B$ approximates ΔW

A is $d \times r$, B is $r \times d$, r is tiny

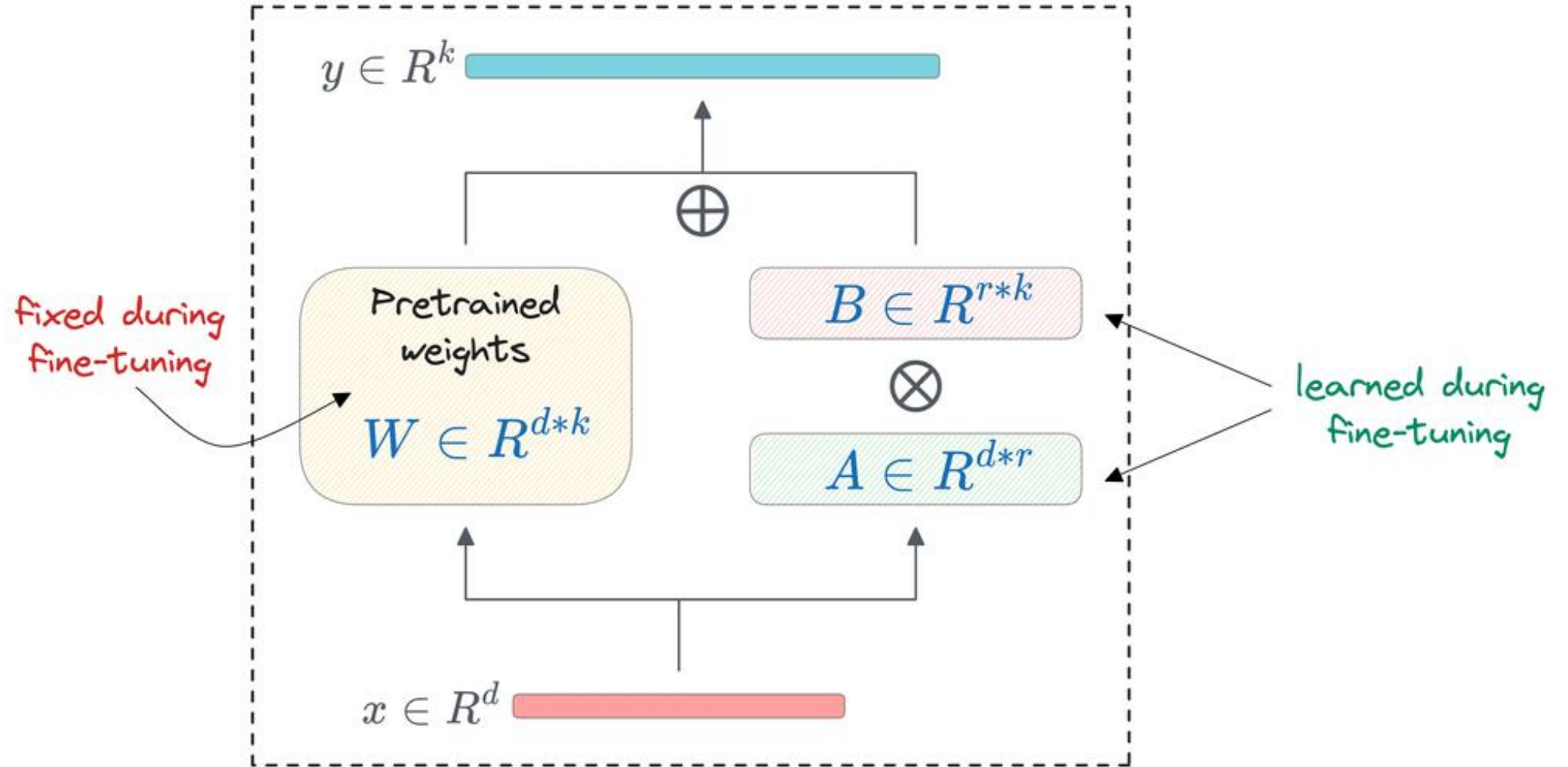


r is tiny. This is all we train.

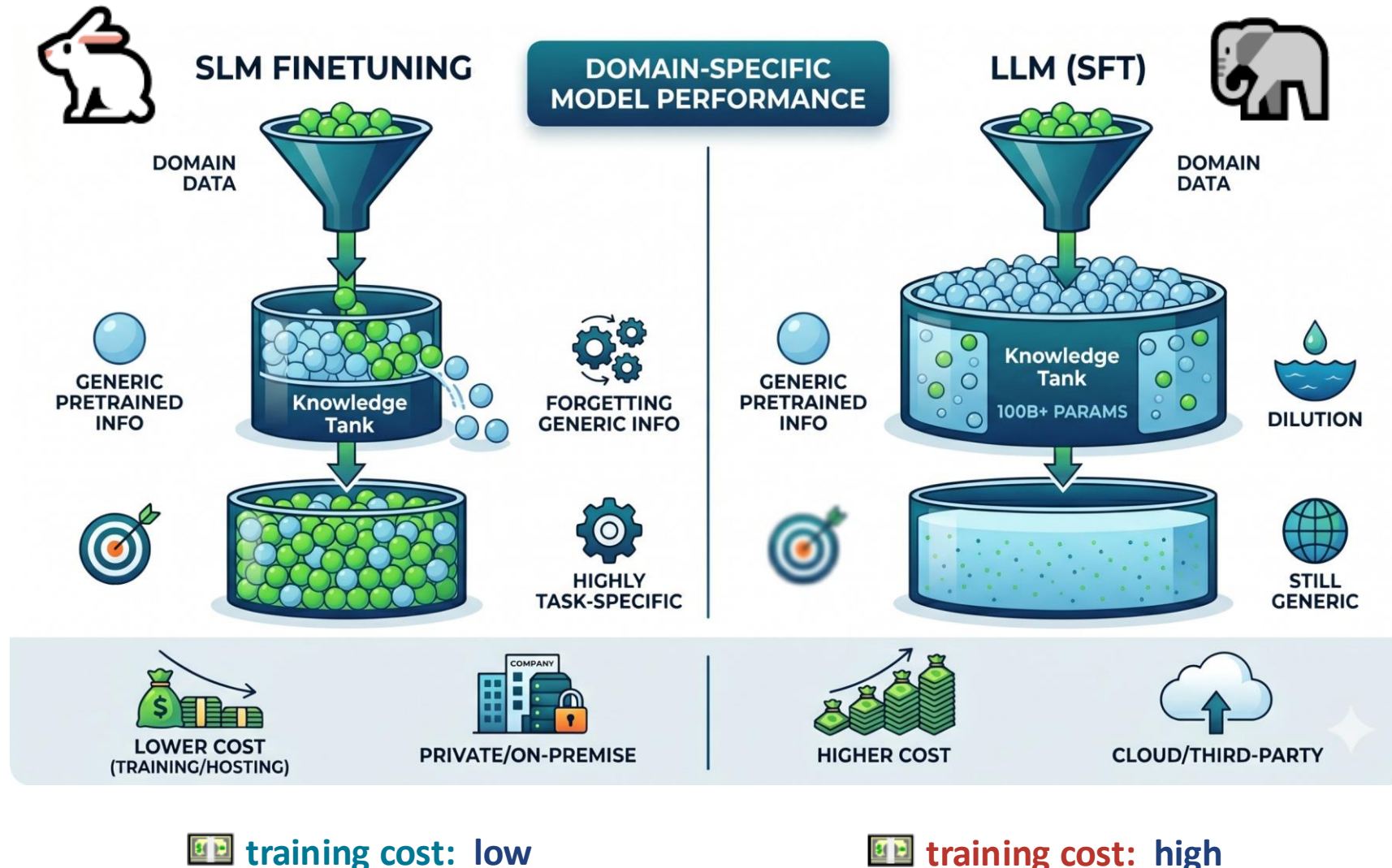


same direction, lightweight. ✓
fraction of the parameters

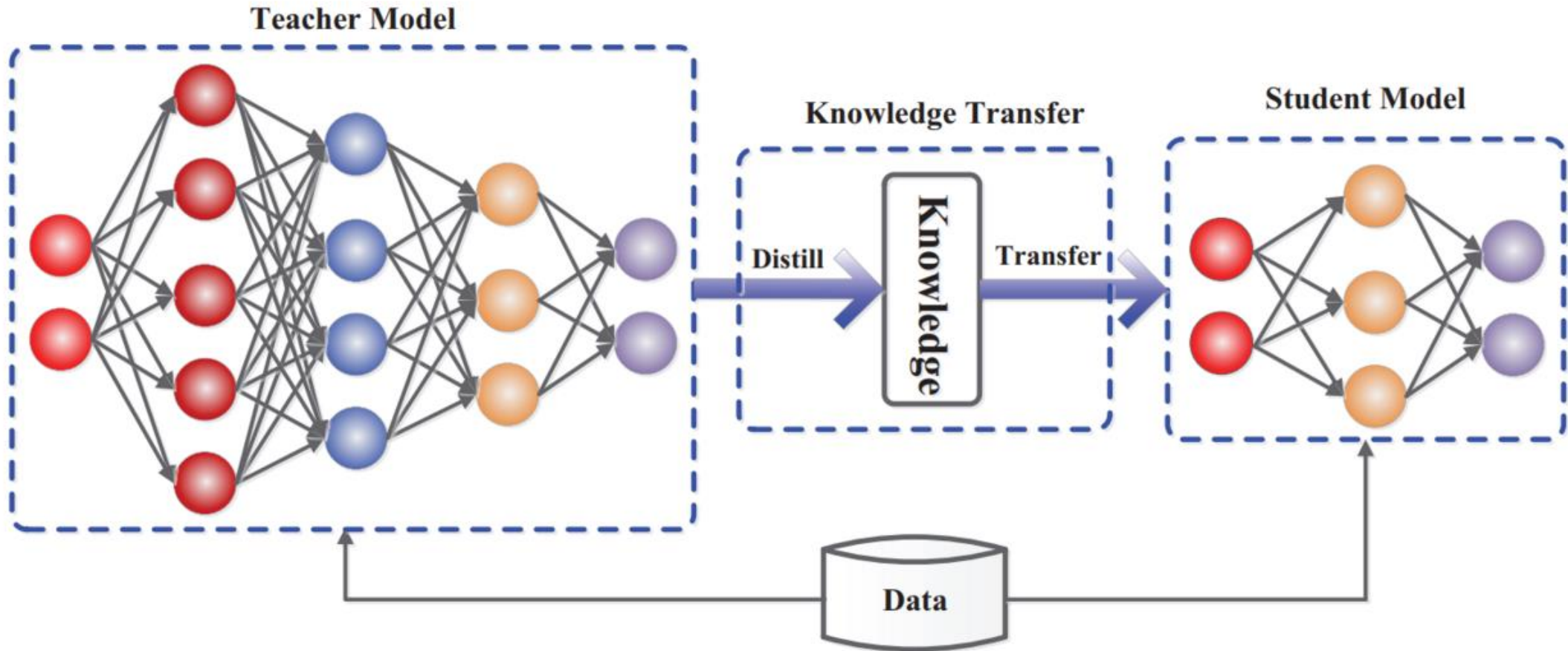
PEFT and LoRA



(S)mall LM vs (L)arge LM



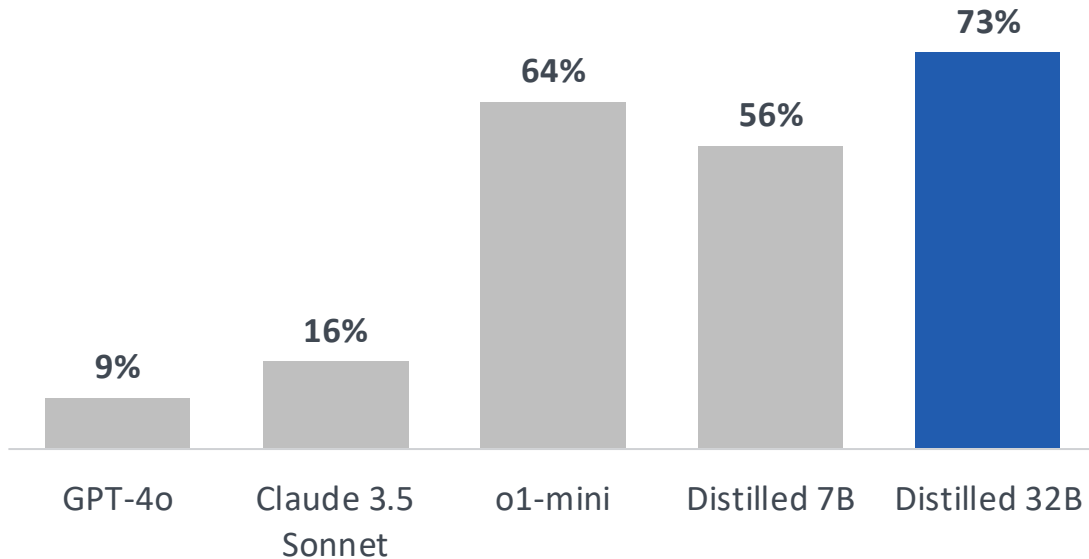
Distillation: the big model teaches the small one



Distilled small models now match the giants

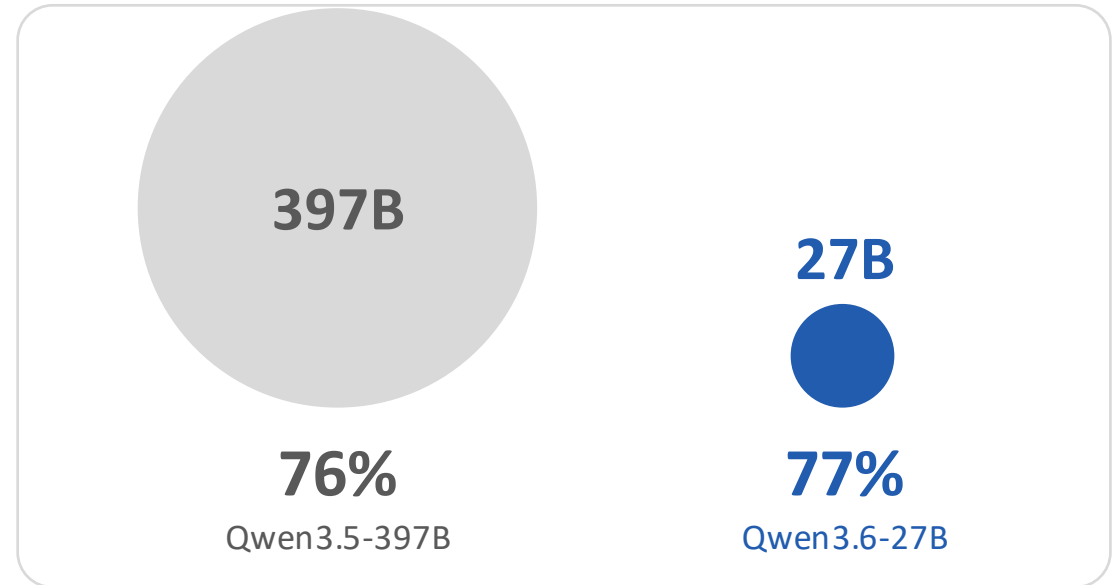
Maths exam score, AIME 2024

DeepSeek-R1 students vs the paid models of 2024



Coding score, SWE-bench Verified

circle area = model size, 2026



The 32B student beats o1-mini at maths; a 27B model matches one 15× its size at coding. For a defined task, the biggest model is rarely needed.

Check before you copy: a September 2026 US advisory names DeepSeek and Alibaba among firms accused of distilling US models against their terms of use. Read the model licence and T&C before you distil or deploy.

Leading small language models in 2026

Model family	Maker	Sizes	Licence	Known for
Phi-4, Phi-4-mini	Microsoft	3.8B – 14B	MIT	Reasoning and maths for its size
Gemma 4	Google	2B – 27B	Gemma terms	Runs on laptops and phones
Qwen3 / 3.5 / 3.6	Alibaba	0.6B – 27B	Apache 2.0	Multilingual and coding all-rounder
Llama 3.1 / 3.2	Meta	1B – 8B	Llama licence	Largest fine-tuning ecosystem
Minstral 3, Mistral Small	Mistral AI (France)	3B – 24B	Apache 2.0	European vendor, fast responses
SmolLM3	Hugging Face	3B	Apache 2.0	Fully open, training data published

All six can be downloaded and hosted on your own servers.

So do we still need the big model?



YES — as the teacher

You cannot distil from nothing. Somebody's frontier model has to write the training data. Your small model is always downstream of a large one.



YES — for the cold start

Before you have any labelled examples, the big model is the only thing that can produce them.



YES — as the escape hatch

Route the routine work to the small model and escalate the rest. NVIDIA's case studies put the replaceable share at 60 % to 70 %.



NO — for the repetitive middle

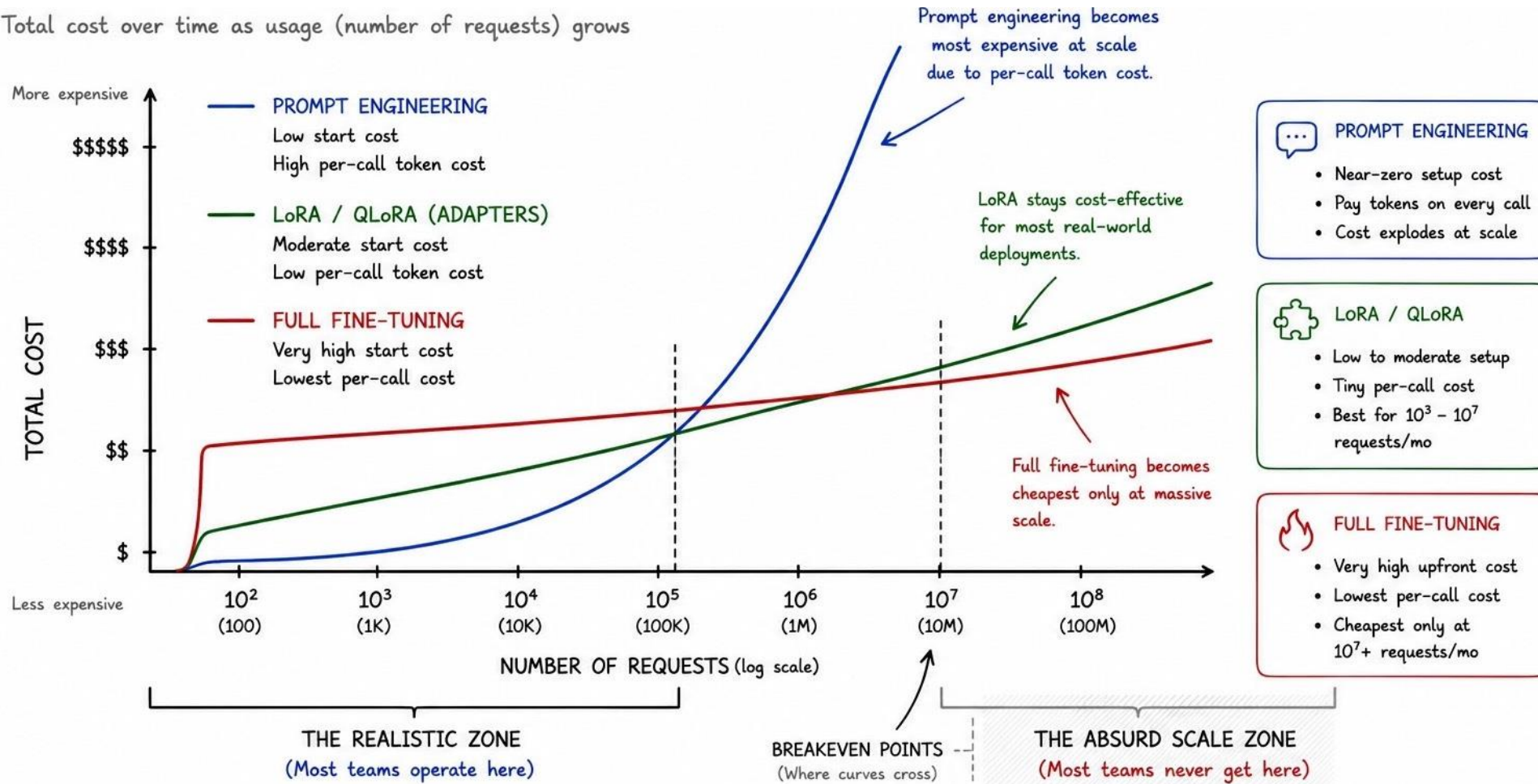
Serving a 7B model costs 10–30× less than a 70–175B one in latency, energy and compute.

Fine-tuning is a one-off project; inference is the running bill

	 FINE-TUNING <i>teaching the model: like training a new hire</i>	 INFERENCE <i>using the model: like paying that hire's salary</i>
What happens	The model studies your examples and updates itself	The model answers a question; it learns nothing. Prompting and RAG need only this step.
How often	Once, and again when the base model is replaced	Every call, every user, every day
Hardware	Rent one GPU for a few hours, then switch it off	GPUs for fast answers at scale; it also runs on a CPU, just slowly
Closed or open	Both work: a provider's API (e.g. OpenAI fine-tuning) or an open model on our GPUs	Both work: the provider's API, or an open model hosted on our own servers
What decides	Data privacy: with an API, our training data goes to the provider	Request volume and data privacy: high volume or sensitive data favours self-hosting

Own it or rent it: the cost curve over time

Total cost over time as usage (number of requests) grows



TAKEAWAY

Prompting is cheap to start but expensive forever.

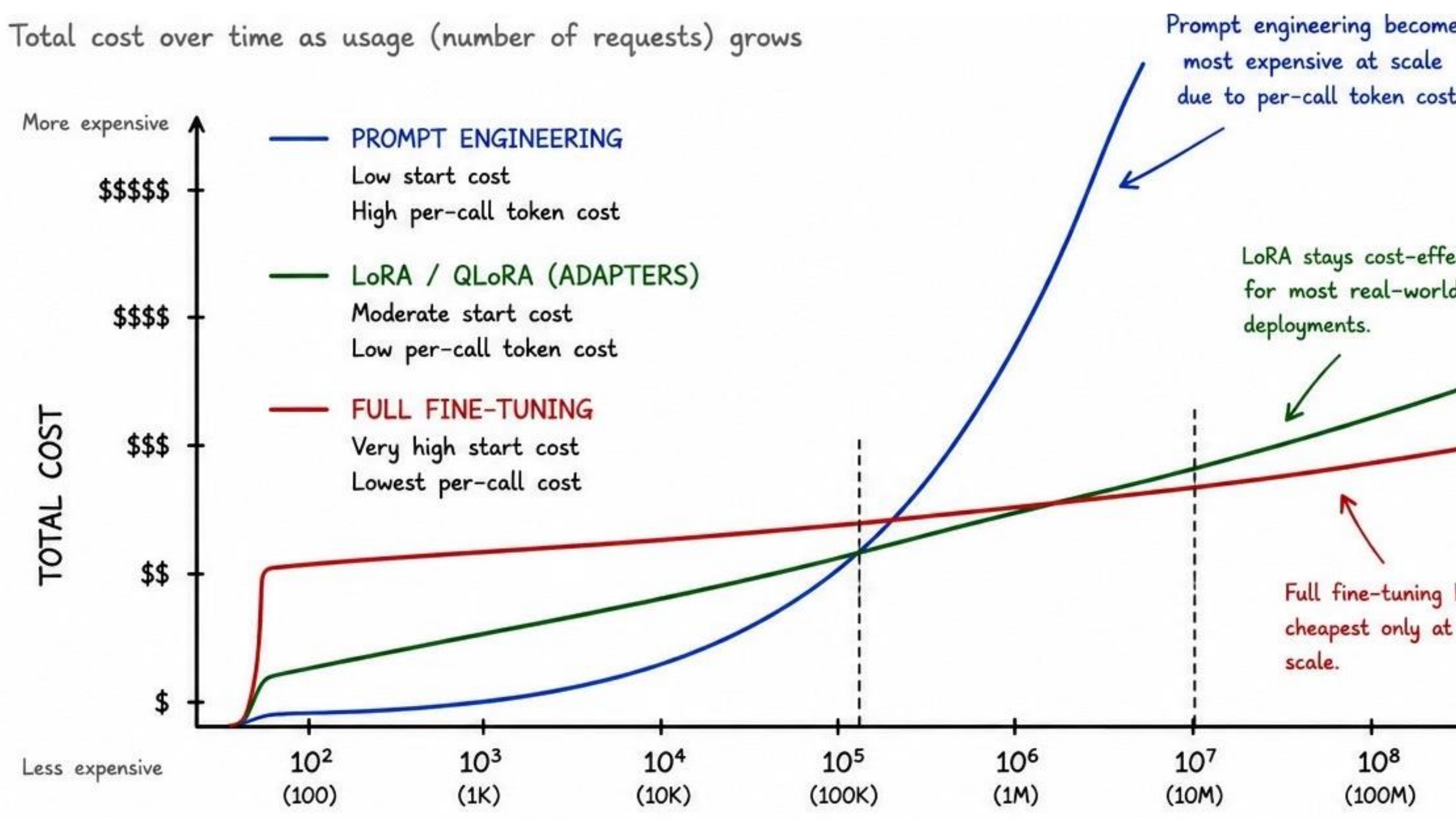
LoRA hits the sweet spot for most products.

Full fine-tuning only wins at massive scale - a threshold most teams never reach.



Don't optimize for the extreme right.
Optimize for where you actually operate.

Total cost over time as usage (number of requests) grows



number of requests) grows

PROMPT ENGINEERING

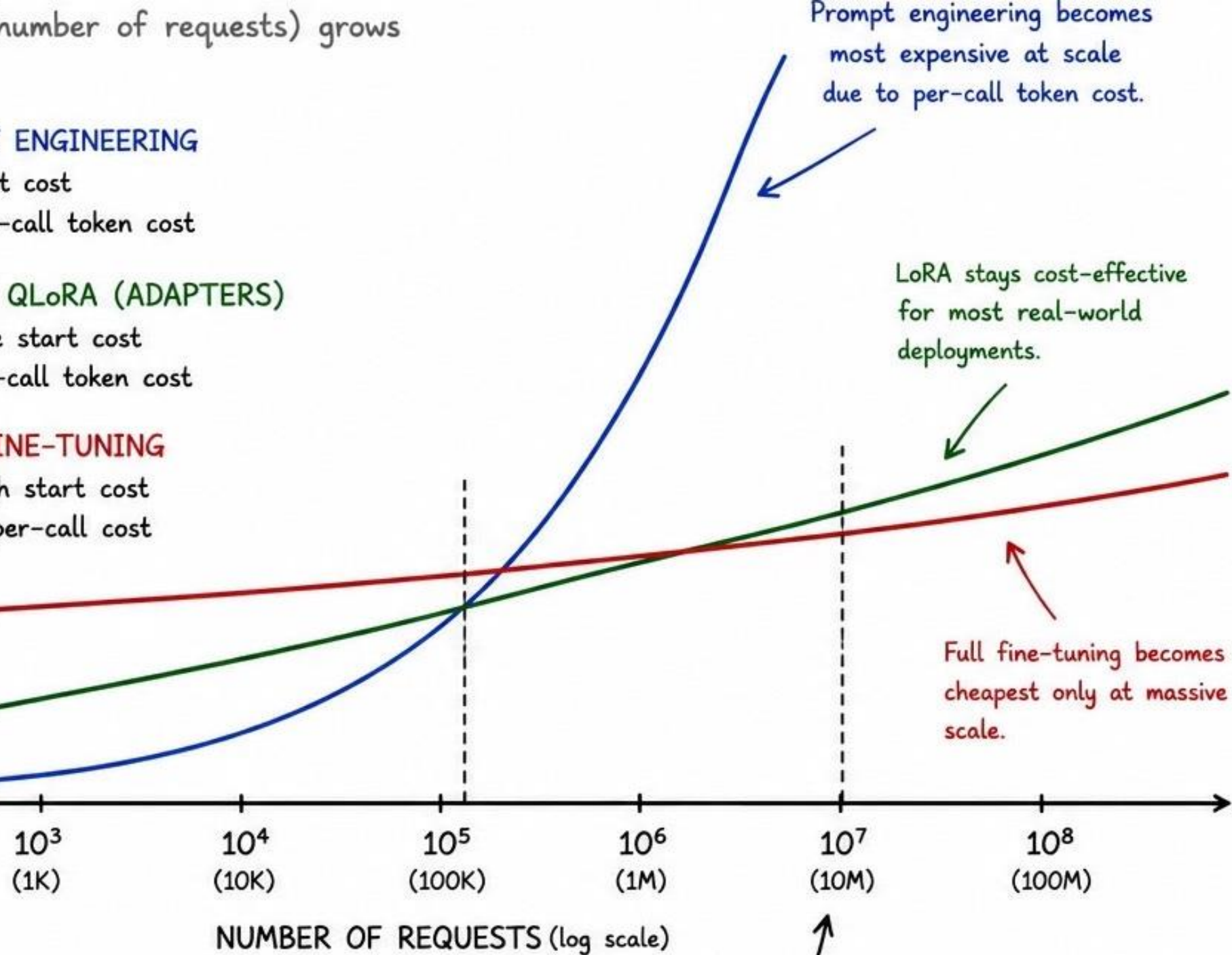
Low start cost
High per-call token cost

QLoRA (ADAPTERS)

Low start cost
Low per-call token cost

FULL FINE-TUNING

High start cost
Low per-call cost



PROMPT ENGINEERING

- Near-zero setup cost
- Pay tokens on every call
- Cost explodes at scale



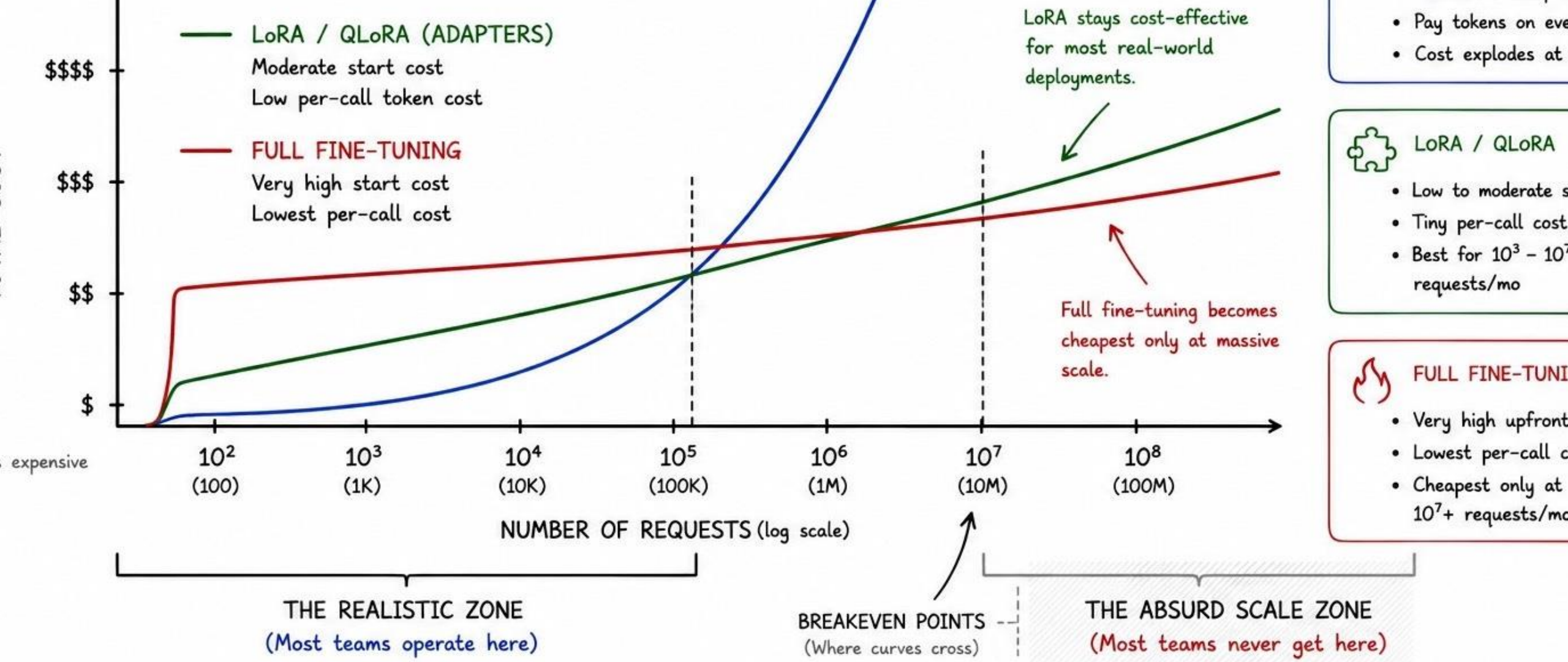
LoRA / QLoRA

- Low to moderate setup
- Tiny per-call cost
- Best for $10^3 - 10^7$ requests/mo



FULL FINE-TUNING

- Very high upfront cost
- Lowest per-call cost
- Cheapest only at 10^7+ requests/mo



TAKEAWAY

Prompting is cheap to start but expensive forever.

LoRA hits the sweet spot for most products.

Full fine-tuning only wins at massive scale - a threshold most teams never reach.



Don't optimize for the extreme right.
Optimize for where you actually operate.

Mixtures of Experts (MoEs)

- Same pipeline as before: pretraining, then fine-tuning inside post-training.
- Change: instead of one dense network, it is split into many smaller finetuned sub-networks called **experts**
- For each word a small **router** activates only a few of them; so the model is huge but stays cheap to run.



MISTRAL AI
Mixtral 8×7B

8 experts per layer, only 2 used per word.



DEEPSEEK
DeepSeek-V3

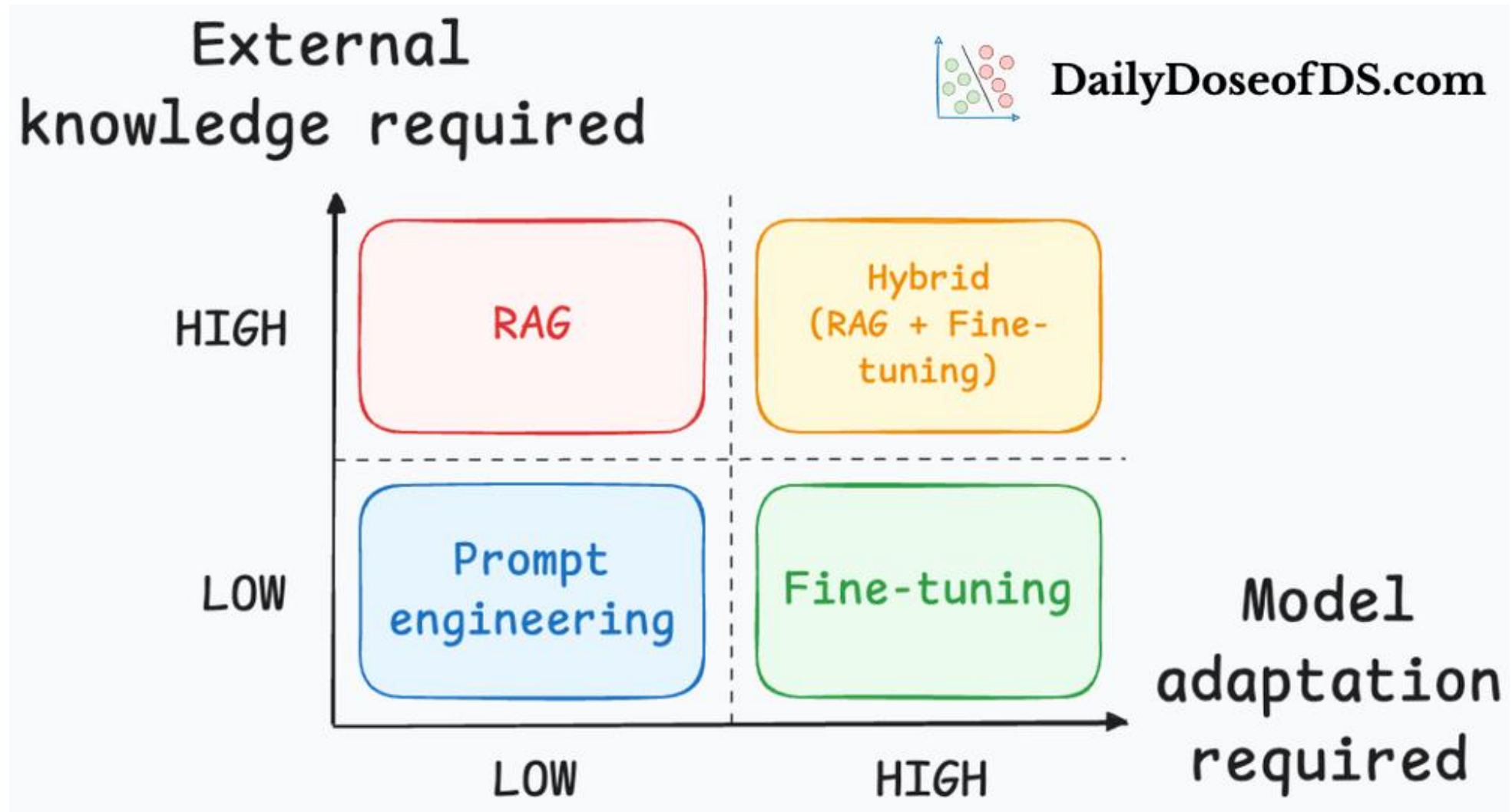
671B parameters, only ~37B active per word.



META
Llama 4

Scout & Maverick — Meta's first MoE models.

When fine-tuning is worth it



Real cases



FINANCE

BloombergGPT

A 50B-parameter model pre-trained from scratch on 363B tokens of proprietary financial data. ~\$10m.



LEGAL

Harvey

Fine-tuned on 10B tokens of US case law. 1,300 organisations and 100,000+ lawyers by 2026.



HEALTHCARE

Med-PaLM 2

Fine-tuned by Google. First model to reach expert level on USMLE-style questions.



FINANCE

Fin-E5

An embedding model for finance search, fine-tuned from the open e5-mistral-7b. Ranks first on FinMTEB.

1. [Bloomberg, 2023](#)
2. [Harvey, 2023](#)
3. [Google Research, 2023](#)
4. [Tang & Yang, FinMTEB, arXiv 2502.10990, 2025](#)

Case 1 — BloombergGPT: the expensive way

FINANCE

Build a finance model from scratch, on data nobody else has.



THE DATA

363B

tokens from Bloomberg's own 40-year archive, plus 345B of general text



THE METHOD

\$10M

full pre-training from scratch: 50B parameters, 1.3M GPU-hours

Model: BloombergGPT (own, BLOOM-style)



THE RESULT

43% vs 30%

on financial Q&A (ConvFinQA) against a comparable open model



TODAY

~1 year

until GPT-4 beat it without Bloomberg's data. No successor shipped.

Case 2 — Harvey: the way that worked

LEGAL

Fine-tune on top of somebody else's frontier model, on data you are allowed to use.



THE DATA

10B

tokens of public US case law, curated with OpenAI. No client files.



THE METHOD

Fine-tune

on top of a frontier model, trained to cite the case behind every sentence

Model: OpenAI GPT-4



THE RESULT

97%

of the time, lawyers preferred its answers. 83% more factual than GPT-4.



TODAY

100,000+

lawyers at 1,300 organisations; valued at ~\$11bn in 2026

Case 3 — Med-PaLM 2 → Med-Gemini: the moving target

HEALTHCARE

What it looks like when the last few percentage points are the ones that matter.



THE DATA

0

raw patient records: public medical exams plus de-identified clinical text



THE METHOD

FT + search

instruction fine-tuning, later combined with web search when the model is unsure

Model: PaLM → PaLM 2 → Gemini



THE RESULT

67% → 91%

on the US medical licensing exam (MedQA) in under two years



TODAY

3 models

Flan-PaLM, Med-PaLM 2, Med-Gemini: every gain needed a new training run

Case 4 — Fin-E5: proof you can check

FINANCE

Fine-tune an open embedding model for finance search, then publish the benchmark.



THE DATA

19,467

finance training examples,
generated synthetically and
all published



THE METHOD

Fine-tune

as an embedding model, in
about 100 training steps

Model:
e5-mistral-7b-instruct
(open, 7B)



THE RESULT

#1

on FinMTEB (64 finance
datasets), ahead of paid
commercial APIs and 4.5%
above its base model



TODAY

Open

weights are free on Hugging
Face, and the benchmark
lets anyone check the claim

Four ways to make AI work on your own data

More effort and cost, more control over the answers



1 Prompt with your documents

Paste the relevant files into the question. Fine for a handful of documents.



2 RAG: search, then answer

The system finds the best pages in your library and hands them to the model.



3 RAG + custom search model

The search step is trained on your jargon, so it finds the right page.



4 RAG + custom search + fine-tuned model

The model also learns your format, tone and task. Search still supplies the facts.